Ontology for Payments and Merchant Applications

A Context Graph Architecture for PesaSwap

Bitemporal Data + Decision Traces + AI Agents

Version 1.0 | May 2026 | PesaSwap Engineering

References:
- Foundation Capital "Context Graphs: AI Trillion-Dollar Opportunity" (2025)
- Li Ding "On Boosting Semantic Web Data Access" AAAI-05
- W3C Web Payments Ontology
- Palantir Foundry Ontology Architecture
- Schema.org Standard Vocabularies

1. Executive Summary

This document defines a formal ontology for PesaSwap, integrating:

- Semantic Web ontology (RDF/OWL) for entity modeling and interoperability (AAAI-05)
- Context Graphs for searchable decision traces (Foundation Capital 2025)
- Palantir Foundry three-layer ontology (semantic, kinetic, dynamic)

The ontology transforms PesaSwap from a payment processor into an AI-native intelligence platform where every interaction becomes searchable context.

Litmus test: "Can your system tell me what a specific agent knew at 2:14 PM last Tuesday when it made a specific decision?"

2. Theoretical Foundations

2.1 Semantic Web & RDF (AAAI-05)

Li Ding's Web of Belief (WOB) ontology models three interactive worlds: the Web, the RDF graph world, and the agent world. Key principles applied:

- Agents discover, consume, and evaluate data quality before reasoning
- Ontology emergence: structure reveals itself from usage patterns
- Trust networks: provenance + trust scores determine fact reliability
- Scalability via metadata: document-level indexes enable discovery at scale

2.2 Context Graphs (Foundation Capital)

Three requirements for context graphs:

- Adopt existing standards for basics (Schema.org). Don't reinvent Person, Org, Event.
- Make time first-class. Store what was true WHEN decisions were made.
- Capture reasoning in real time. WHY exceptions happened, not just THAT they happened.

Value: every exception = training data. Every override = case study. Every trace = faster future decisions.

2.3 Prescriptive vs. Emergent Ontology

- PRESCRIPTIVE: Define core entities upfront (Payment, Customer, MenuItem). Use standards.
- EMERGENT: AI discovers relationships ("window seats = 23% higher spend on Fridays").
- HYBRID (our approach): Bootstrap with standards, learn domain nuances from data.

3. PesaSwap Domain Ontology

3.1 Core Entity Classes

Schema.org foundation with domain extensions:

Class	Extends	Description	Key Properties
Customer	schema:Person	Diner/payer	phone, methods, history
Merchant	schema:Org	Restaurant operator	name, venues, plan
Staff	schema:Person	Employee	role, permissions, shift
MenuItem	schema:Product	Food/drink	price, modifiers, category
Menu	schema:ItemList	Item collection	categories, schedule
Table	schema:Place	Dining position	number, zone, capacity
Zone	schema:Place	Venue area	tables, assignedMenus
Payment	schema:PayAction	Transaction	amount, method, status
Order	schema:Order	Customer order	items, table, total
Refund	schema:Transfer	Reversal	amount, reason, approver
Decision	(custom)	Judgment record	actor, action, reasoning
Agent	(custom)	AI decision-maker	type, model, confidence

3.2 Relationship Types

Relation	Domain->Range	Description
placesOrder	Customer->Order	Customer initiates order at table
processesPayment	Agent->Payment	AI routes payment method
approvedBy	Refund->Staff	Staff authorized the refund
servedBy	Table->Staff	Which staff served this table
belongsToZone	Table->Zone	Physical zone assignment
pairsWellWith	MenuItem->MenuItem	Upsell/linked product
precedentFor	Decision->Decision	Historical reasoning chain
trustsSource	Agent->DataSource	Trust weight for quality

3.3 Standard Properties

- id (URI) - Globally unique identifier
- valid_from, valid_to - Temporal validity window
- recorded_at - When system captured this fact
- source - Provenance (pos_sync, staff, ai_agent, customer)
- confidence - Float 0-1 (observed=1.0, inferred=0.6-0.9)

4. Temporal Architecture (The Clock)

"The hard part isn't the graph. It's the clock." Every fact has two time dimensions enabling: "What did the system know when this decision was made?"

4.1 Bitemporal Fact Model

```
CREATE TABLE facts (
  id            TEXT PRIMARY KEY,
  entity_id     TEXT NOT NULL,
  entity_type   TEXT NOT NULL,
  attribute     TEXT NOT NULL,
  value         JSON NOT NULL,
  valid_from    TIMESTAMP NOT NULL,  -- When true in reality
  valid_to      TIMESTAMP,           -- When stopped (NULL=current)
  recorded_at   TIMESTAMP NOT NULL,  -- When system captured
  source        TEXT NOT NULL,
  actor_id      TEXT,
  confidence    REAL DEFAULT 1.0,
  superseded_by TEXT
);
```

4.2 Event Clock Queries

```
-- "What was the price on Feb 20 at 14:14?"
SELECT value FROM facts
WHERE entity_id = 'item-nyama-choma' AND attribute = 'price'
  AND valid_from <= '2026-02-20T14:14:00Z'
  AND (valid_to IS NULL OR valid_to > '2026-02-20T14:14:00Z')
ORDER BY recorded_at DESC LIMIT 1;

-- "Who had authority when refund was issued?"
SELECT value FROM facts
WHERE entity_id = 'staff-james' AND attribute = 'role'
  AND valid_from <= '2026-05-31T14:22:00Z'
  AND (valid_to IS NULL OR valid_to > '2026-05-31T14:22:00Z');
```

4.3 Critical Temporal Questions

- What price was charged vs. what SHOULD have been? (disputes)
- Who had authority at the time of this override? (audit)
- What menu was active when customer ordered? (complaints)
- Was item sold-out before or after the order? (timing)

5. Decision Trace Layer

Captures WHY decisions were made, not just WHAT happened. Transforms exceptions into searchable precedent.

5.1 Decision Trace Schema

```
CREATE TABLE decisions (
  id              TEXT PRIMARY KEY,
  timestamp       TIMESTAMP NOT NULL,
  actor_type      TEXT NOT NULL,    -- "human" | "ai_agent"
  actor_id        TEXT NOT NULL,
  actor_role      TEXT,
  action          TEXT NOT NULL,
  outcome         JSON NOT NULL,
  trigger         TEXT,
  context_snapshot JSON,
  precedents_used JSON,
  rules_applied   JSON,
  rules_overridden JSON,
  justification   TEXT,
  confidence      REAL DEFAULT 1.0,
  entities_involved JSON NOT NULL,
  supersedes      TEXT
);
```

5.2 Example: Refund Decision

```
{
  "action": "approve_refund",
  "actor": "staff-james", "role": "shift_manager",
  "outcome": {"amount": 500, "currency": "KES"},
  "trigger": "Food cold after 25min wait",
  "context": {"kitchen_staff": 2, "normal": 4, "wait": 25, "avg": 12},
  "precedents": ["dec-05-15-003", "dec-04-22-007"],
  "rules_applied": ["mgr approves up to KES 1000"],
  "justification": "Understaffed kitchen. Wait 2x avg. Loyal customer (8 visits)."
}
```

5.3 Precedent Search

```
SELECT id, justification, outcome
FROM decisions
WHERE action = 'approve_refund'
  AND context_snapshot->>'kitchen_staff_count' < 3
ORDER BY vector_distance(embedding, query_embedding) ASC
LIMIT 5;
```

Result: "In 3 similar cases, managers approved 50-100% refunds. Recommend: approve KES 500."

6. Agent Architecture

Agents sit in the execution path, capturing reasoning as byproduct of work.

6.1 Payment Decision Agent

- Routes to optimal method (M-Pesa STK, card, saved method, widget)
- Assesses fraud risk from customer history graph + temporal patterns
- Records trace: why this method, confidence, precedents used
- Context: customer graph + current request + temporal (time, day, load)

6.2 Menu Intelligence Agent

- Discovers: "Pilau orders +40% when raining" (emergent pattern)
- Optimizes category order from conversion data
- Recommends pairings from actual co-purchase graph (not manual)
- Predicts sold-out risk from consumption rates

6.3 Service Quality Agent

- Monitors order-to-serve time vs zone average
- Correlates: "James on terrace = 18% tips vs 12% baseline"
- Predicts complaint risk (long wait + understaffed)

6.4 Fraud & Anomaly Agent

- Card testing detection (5x in 10min across tables)
- Unusual refund patterns (staff <-> customer collusion)
- Split-payment gaming (threshold avoidance)

7. Data Quality & Trust

From AAAI-05 Web of Belief: not all sources equal. Agents evaluate before reasoning.

7.1 Trust Scores

```
Source                          Score
POS System (direct sync)        0.95
Staff Manual Entry              0.85
Customer Self-Report            0.70
AI Agent Inference              0.60-0.90
Third-Party API                 0.80

Conflicts: higher trust wins. No winner: flag for human review.
```

7.2 Quality Dimensions

- ACCURACY - Correct? Validated against ground truth
- COMPLETENESS - All required attributes present?
- TIMELINESS - Current or stale?
- CONSISTENCY - Contradicts other facts?
- PROVENANCE - Traceable origin?
- TRUST - Reliable source?

8. Technical Stack

Layer	Technology	Purpose
Frontend	React 19 + TanStack	Already built, proven at scale
Edge	Cloudflare Workers	Global, <10ms, auto-scale
Graph	SurrealDB	Entity relations, traversal, patterns
Temporal	Cloudflare D1	Bitemporal facts, ACID, decisions
Vector	CF Vectorize	Semantic precedent search
AI	Workers AI + Claude	Agent reasoning, discovery
Auth	Clerk	Multi-tenant, RBAC, SSO
Payments	PesaSwap SDK	M-Pesa, cards, wallets
Storage	Cloudflare R2	Images, PDFs, media
Queue	CF Queues	Async trace writes

API Endpoints

```
GET  /api/v1/entities/:id/at?timestamp=...   (point-in-time query)
GET  /api/v1/entities/:id/history            (temporal history)
POST /api/v1/decisions                       (record decision)
GET  /api/v1/decisions/search?similar_to=... (precedent search)
POST /api/v1/agents/payment/decide           (payment routing)
POST /api/v1/graph/query                     (graph traversal)
```

9. Phased Delivery Plan

Phase 1: Temporal Foundation (Weeks 1-2)

- Deploy D1 bitemporal schema (entities, facts, decisions, relationships)
- Temporal middleware: every mutation auto-records provenance
- Migrate localStorage to D1 with temporal columns
- Core query: getEntityStateAt(id, timestamp)
- Auth (Clerk) with multi-tenant isolation

Phase 2: Decision Traces (Weeks 3-4)

- Decision trace capture middleware
- Staff reasoning UI (refund reason, override justification)
- Precedent search API (structured + vector)
- Payment Agent v1: routes based on customer graph
- Vectorize deployment for semantic search

Phase 3: Graph Intelligence (Weeks 5-6)

- SurrealDB for entity relationship graph
- Import pipeline: D1 facts -> graph edges
- Menu Agent: co-purchase patterns, demand prediction
- Service Agent: staff-outcome correlations
- Dashboard AI Insights page

Phase 4: Trust & Quality (Weeks 7-8)

- Trust scoring for all data sources
- Provenance explorer UI
- Fraud Agent v1: anomaly detection
- Conflict resolution with human escalation
- Full audit replay capability

TOTAL: 8 weeks (2 engineers) | COST: $80-215/month at 100K users

VALUE: Every interaction becomes compounding intelligence. The system gets smarter with every payment, every order, every decision.

10. Appendix: OWL/RDF Definitions

Core ontology in Turtle (RDF) format:

```
@prefix ps: <https://ontology.pesaswap.io/> .
@prefix schema: <https://schema.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

ps:Customer a owl:Class ; rdfs:subClassOf schema:Person .
ps:Merchant a owl:Class ; rdfs:subClassOf schema:Organization .
ps:Staff a owl:Class ; rdfs:subClassOf schema:Person .
ps:MenuItem a owl:Class ; rdfs:subClassOf schema:Product .
ps:Payment a owl:Class ; rdfs:subClassOf schema:PayAction .
ps:Order a owl:Class ; rdfs:subClassOf schema:Order .
ps:Decision a owl:Class ; rdfs:subClassOf schema:Action .
ps:Agent a owl:Class ; rdfs:subClassOf schema:SoftwareApplication .

ps:validFrom a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:validTo a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:recordedAt a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:confidence a owl:DatatypeProperty ; rdfs:range xsd:float .

ps:placesOrder a owl:ObjectProperty ;
  rdfs:domain ps:Customer ; rdfs:range ps:Order .
ps:processesPayment a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:Payment .
ps:approvedBy a owl:ObjectProperty ;
  rdfs:domain ps:Decision ; rdfs:range ps:Staff .
ps:precedentFor a owl:ObjectProperty ;
  rdfs:domain ps:Decision ; rdfs:range ps:Decision .
ps:trustsSource a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:DataSource .
```

This ontology is extensible. New classes and properties can be added as agents discover patterns. The bitemporal model tracks ontology evolution itself.